EEE3095/6S Final Exam Solutions

November 2024

Duration: 2 hours Total: 85 marks

Empl ID: _____ Student Num: _____

Instructions:

- This is a closed-book exam.
- There are **three** Sections (A, B and C) to this exam.
- You must use a pen to complete the exam – pencil will only be marked for drawings. Values indicated on drawings must be written in pen.
- Please show all your working clearly – your method and approach matter.
- Keep all your answers within the allocated blocks; anything outside of these blocks will not be marked.
- A reference sheet (which you may detach) appears at the end of this paper.

Question	Available Marks	Grade
1	9	
2	9	
3	9	
4	8	
5	9	
6	4	
7	4	
8	19	
9	5	
10	9	
Total:	85	

Section A: Multiple Choice

Empl ID: | | | | | | | |

Question:	1	2	3	4	Total
Points:	9	9	9	8	35
Score:					

Question 1 [9 marks]

a. The I2C communication protocol is supported by many components. Which one of the properties below is **not** a characteristic of I2C? (3)

√ **I2C is a full-duplex protocol**
◯ I2C is a serial protocol standard
◯ I2C is a synchronous protocol (i.e. depends on a shared clock of some sort)
◯ I2C is a two-wired protocol

Solution: I2C is a half-duplex protocol, meaning data can only be transmitted in one direction at a time.

b. Consider the following line of Assembly code. What will the GAS assembler do when encountering this code? (3)

```
LDR r12, =0xFFFE0010
```

◯ The assembler will skip the line, treating the = as an indication that the programmer has not yet finalised the instruction.
◯ GAS will generate an error, indicating that the symbol = is unexpected.
◯ GAS will generate an error stating "Error: undefined symbol r12" because r12 is not a valid register name.
√ **GAS will place the value 0xFFFE0010 into a nearby memory location (e.g., 14 bytes away) and replace the instruction with LDR r12, [pc, #14].**

c. What is the purpose of the ST-Link/V2? (3)

◯ Connects to everything on the board and executes code
√ **Interprets USB commands from the computer and converts them to Serial Wire Debug (SWD) commands that can control the micro**
◯ Converts the 5 V provided by the USB port into 3.3 V
◯ Translates between TTL or CMOST logic-level UART traffic and bi-polar higher voltage RS-232 traffic; used for industrial communications links

Question 2 [9 marks]

a. Generally, for our micro, by what base-10 value is the Program Counter incremented on every clock cycle? (3)

◯ 8
◯ 1
◯ 16
√ **2**

Solution: Each instruction is generally 16 bits (2 bytes) wide, so the PC must be incremented by a decimal value of 2 to get to the next instruction.

b. Which one of the following is not a characteristic of a Reduced Instruction Set Computing (RISC) architecture? (3)

◯ Emphasis on software to perform complex operations
◯ Different addressing modes
√ **Instructions have different lengths**
◯ Load/store architecture

c. Select the incorrect statement below regarding ARM Assembly statements... (3)

◯ Instructions can modify registers r0 to r12 regardless of which mode is active.
√ **The ARM *in* and *out* instructions respectively receive inputs or send output values using only registers r0 to r3.**
◯ It is possible to modify the contents of the link register (lr), but it could cause problems when returning from function calls.
◯ At the start of a function (e.g. called using *bl* branch and link), there is no facility to check the status of the CPSR to determine the current mode.

Solution: The statement B is incorrect, as the in and out instructions are not part of the standard ARM instruction set. They are typically used in x86 assembly language for input/output operations. In ARM, as you should well know by now, input/output is usually handled through memory-mapped I/O or specific peripheral registers.

Question 3 [9 marks]

a. What type of state machine is shown in Figure 1 below? Select only one option. (3)

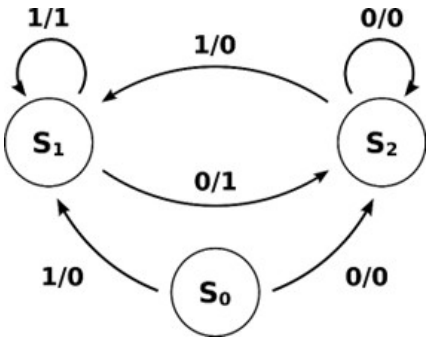


Figure 1: Mystery type of machine

◯ Moore
◯ Monroe
√ **Mealy**
◯ Mandalay

b. You are given the C function below, and you may assume that an optimising GCC compiler is used (e.g., `gcc -O1`): (3)

```
int do_something ( int x ) {
    if (x>0) return x-100; else return x+100;
}
```

Which one of the following Assembly snippets is a correct translation of this function to 32-bit ARM Assembly?

◯
```
do_something:
  cmp     r0, #100
  addgt   r0, r0, #1
  addle   r0, r0, #-1
  bx      lr
```

◯
```
do_something:
  tst     r0, #1
  subne   r0, r0, #100
  addeq   r0, r0, #100
  bx      lr
```

√
```
do_something:
  cmp     r0, #0
  subgt   r0, r0, #100
  addle   r0, r0, #100
  bx      lr
```

◯
```
do_something_else:
  cmp     r1, #0
  addle   r0, r0, #100
  subgt   r0, r0, #100
  brx     lr
```

Solution: If you look closely, particularly at the first instructions, you can immediately eliminate likely cases. It is a compare x>0 involving r0. It isn't a bit test either, so that leaves C as most likely, and indeed looking more at the return values, the correct answer.

c. You are involved in designing a single-cycle CPU. The processor has the following stages with their respective propagation delays in parentheses: (3)

1. PC Increment (1 ns)
2. Instruction Fetch (5 ns)
3. Register Read (1 ns)
4. Read Data Forwarding (1 ns)
5. Function Unit/Scratch Access (4 ns)
6. Write Data Forwarding (1 ns)
7. Register Write (1 ns)

A colleague in marketing, who is less familiar with processor design, insists that the processor can operate at 200 MHz (1/5 ns), the speed of its slowest stage.

Which of the following clock speeds would be a more suitable recommendation for this processor?

◯ 200 MHz (1/5 ns) - As per your colleague's suggestion.
◯ 100 MHz (1/10 ns) - Half the time of the slowest stage for safety.
√ **50 MHz (1/20 ns) - A commonly used speed and slightly under the theoretical maximum.**
◯ 31.25 MHz (1/32 ns) - A convenient multiple of 2.

Solution: Explanation of incorrect options:

A: The colleague's suggestion is incorrect because it only considers the slowest stage and ignores the cumulative delay of all stages.

B: While halving the time of the slowest stage provides some margin, it might still be too aggressive, as it does not account for potential variations in propagation delays due to environmental factors or manufacturing tolerances.
D: While the period for 31.25 MHz is a convenient multiple of 2, it's likely too conservative and would result in underutilising the processor's potential.

Why C is the best choice:

- Total Delay Consideration: The total propagation delay for all stages is 14 ns. To ensure reliable operation, the clock period should be slightly longer than this to allow for setup and hold times, as well as potential variations in delays.
- Realistic Clock Speed: 50 MHz (1/20 ns) provides a reasonable margin over the calculated minimum clock period while still achieving a decent performance level.
- Common Speed: 50 MHz is a commonly used clock speed, which can simplify interfacing with other components and reduce design complexity.

Question 4 [8 marks]

a. Complete Table 1 below by matching each term in the left column with an appropriate description in Table 2. You need only fill in the number corresponding to each description. (5)

Term	Description
!SS	_____
Data type	_____
Baud rate	_____
CPOL	_____
CPHA	_____

Table 1: Answers table

No.	Description
#1	Determines how much space will be allocated to a variable
#2	Determines the CLK logic level while idling
#3	Determines the slave device with which to communicate
#4	Determines whether we sample on the first or second CLK edge
#5	Determines asynchronous communication speed

Table 2: Descriptions to choose from

Solution:

Term	Description No.
!SS	#3
Data type	#1
Baud rate	#5
CPOL	#2
CPHA	#4

b. The machine code format of the `LDR` instruction has 6 bits of opcode, 5 bits of register specifications, and 5 bits for an immediate offset. True or false? (1)

◯ True √ **False**

c. A 10-bit signed number can represent values from -511 to 512. True or false? (1)

◯ True √ **False**

d. The ARM Cortex-M0 implements what is known as a three-stage pipeline, implementing the operations of Fetch-Encode-Execute. True or false? (1)

◯ True √ **False**

Section B: Short Answers

Empl ID: | | | | | | | |

Question:	5	6	7	Total
Points:	9	4	4	17
Score:				

Question 5 [9 marks]

a. C is a very popular, powerful and flexible programming language that is often called a "portable Assembly language". Answer the following questions:

(i) In what way is C seen as "portable Assembly" compared to, for example, Python or Java? (2)

Solution: C offers a very low level of abstraction compared to Java/Python/etc. and is known for being extremely close to the hardware, offering features such as direct memory manipulation. The aspect of portability relates to the same, or very similar, C code being able to do much the same thing on different architectures although the lower level machine instructions it traslates to may be very different between the architectures on which the code compiled for and run on.

(ii) A feature of C is its support for pointers. Consider an array of integers set up as a local variable, using "int myarray[10];". Explain the behavior of the following code and the output it would produce when run on a PC (i.e., a platform that has a screen on which `stdio` can display things). (4)

```
int myarray[10];
*myarray = 99;
printf("\%d:\%d\n",(int)*myarray,(int)\&myarray);
```

(Hint: Note that the second value printed by printf might not be a decimal number... and recall that myarray[0] is equivalent to *(myarray + 0).)

Solution: myarray creates a 10-index array of integers (but you don't have to say that since it is very applied in the question). *myarray directly dereferences the pointer that myarray decays into, accessing the first element (equivalent to myarray[0]). `(int)&myarray` does a type cast of (int*), the data type of myarray, to integer type. The printf hence prints the first value of myarray (that is 99) as well as the address of myarray as in integer. Thus, the printf would return two decimal numbersm. I didn't ask for a %p to print the pointer value out, so the second number would likely be some big number (but probably not negative because it's unlively the full 64bits of memory space are going to be fully populated with RAM). So output might be something like: `99:6356696`

(iii) What would happen if the following command was successfully run in a terminal? (Note: assume main.c is a correctly written C program that uses libraries available on the platform it is being compiled on.) (3)

```
gcc -O1 main.c -o myprog
```

Solution: This would:

- compile the main.c file using the GCC compiler
- store the output executable binary in a file called "myprog"
- apply compiler optimisation level 1 (basic optimisations that do not take too long)

Question 6 [4 marks]

a. If we have the data 0xAB that needs to be transmitted with a parity bit appended, what would be the final data word that is transmitted for both odd and even parity? (2)

Solution: The 8-bit data is 0xAB, which corresponds to 0b10101011; this contains a total of 5 ones, so:

- for odd parity, the full 9-bit data would be 0b101010110 (a zero is appended for a total of 5 ones)
- for even parity, the full 9-bit data would be 0b101010111 (a zero is appended for a total of 6 ones)

b. RS-485 uses a "twisted cable" configuration; what is the purpose of this? (2)

Solution: RS-485 use twisted cables so that they pick up less noise (by reducing interference from external electromagnetic fields), and this is achieved by minimising the gap between the two wires.

Question 7 [4 marks]

a. Assume we have: (2)

1. a CPU running at 8 MHz
2. a delay loop which consumes 5 cycles per loop iteration
3. a variable "VALUE" that we want to use to cause a variable delay; VALUE can vary between 0 and 255

If we want the delay to be 0 seconds when VALUE is 0, and 1 second when VALUE is 255, by what number (to the nearest integer) should we multiply VALUE to get the required number of delay loop iterations?

Solution: Each delay loop will take 5/8000000 seconds; so we compute $1/(5/8000000) = 8000000/5 = 1600000$ loop iterations in 1 second, which is when VALUE is 255. Hence, our multiplier M can be calculated using $M \times 255 = 1600000$, or $M = 1600000/255 \approx 6275$.

b. List two reasons why a programmer may want to mix Assembly and C code. (2)

Solution: Any two of the following:

- Interfacing an Assembly program with C libraries (or vice-versa)
- Performance – writing speed-critical key parts of a program in Assembly, and leaving the rest in C, may boost overall performance
- Size – human-written Assembly can be smaller than compiler-generated code
- Doing something very fast and specialised (through Assembly) as a tiny portion of your (larger) C program

Section C: Long Answers

Empl ID: | | | | | | | |

Question:	8	9	10	Total
Points:	19	5	9	33
Score:				

Question 8 [19 marks]

a. Provide C code for the Finite State Machine (FSM) representation given below (see Figure 2). This FSM describes the way that the traffic lights operate for the T-junction illustrated bottom right; in this T-junction there is a one-way road that enters a two-way road. Traffic lights labelled "A" all work in exactly the same way, which controls the traffic on the two-way road. The traffic lights labelled "B" control the flow of cars from the one-way road into the two-way road. The one-way road also has a sensor "S1" that triggers when a car drives over it. (11)

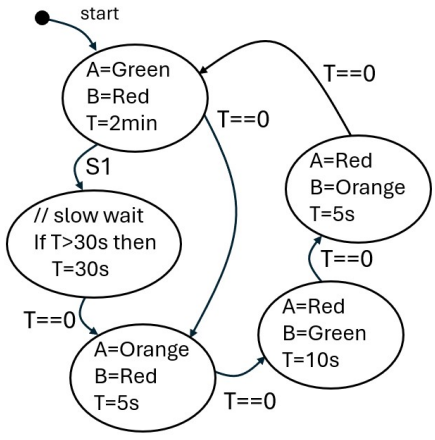
In the FSM, consider that "T" refers to the value of a count-down timer (i.e., T = 10 s sets the timer to count down from 10 seconds), and a check T == 0 is true if the time has counted down to 0; T will remain at 0 when it counters down to 0. The transition labelled "S1" shows that when the S1 sensor is activated, in the first state, operation immediately transitions to the state with comment "slow wait". This slow wait can reduces the time that traffic lights A remain red. It is anticipated that few cars would travel on the one-way road compared to the number travelling on the two-way road.

Note: For your answer, you need to suggest functions or pointers to memory-mapped peripherals that you are using to access times or change the states of the lights on the traffic lights. You can assume, for convenience, that the following constants are already defined:

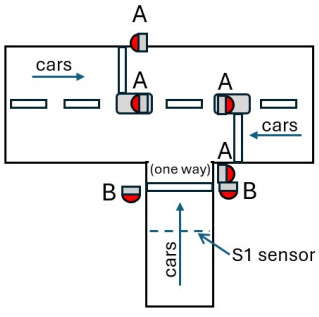
```
@define RED    0x01
@define ORANGE 0x02
@define GREEN  0x03
```

You could, for instance, decide to use functions to control the lights and time using, for example:

```
set_lights_A(GREEN);  // A robots all turn GREEN
set_lights_B(RED);    // B robots all turn RED
set_T(100);       // set count-down timer to 100s
if (get_T()==0) do_something();  // check if timed out
```



Finite state machine for traffic light control



T-junction controlled

Figure 2: T-Junction traffic lights controller

The provision of comments will benefit your marks for this question.

Solution: Sample code solution:

```
// C code for FSM controlling traffic light for T-junction

int state = 0; // define a state variable

// continuously runs the main loop that implements the state machine
while (1) {
switch (state) {
 case 0: // start state
   set_lights_A(GREEN);
   set_lights_B(RED);
   set_T(2*60); // wait in this mode for 2 minutes
   // continue to check S1 until time up
   while (get_T()>0) {
     if (get_S1()) { // check if S1 sensor triggered
       state=1;  // change to slow state if S1 triggered
       break;
       }
     }
   state = 2; // A changes to orange and then red
   break;

  case 1:
    if (get_T()>30) set_T(30); // reduces wait time for B lights
    while (get_T()>0); // just wait for timeout
    state=2;
    break;

  case 2:
    set_lights_(ORANGE);
    set_lights_B(RED);
    set_T(5);
    while (get_T()>0); // just wait for timeout
    state=3;
    break;

  case 3:
    set_lights_(RED);
    set_lights_B(GREEN);
    set_T(10);
    while (get_T()>0); // just wait for timeout
    state=4;
    break;

  case 4:
    set_lights_A(RED);
    set_lights_B(ORANGE);
    set_T(5);
    while (get_T()>0); // just wait for timeout
    state=0; // goes back to start state
    break;
  }  // end of switch
} // end of while
```

b. Imagine you are involved with developing a platform for air quality inspection – the "Air Quality View" (or AQV) device. These are fairly simply units (at least to use) as all it shows is a number from 0.0 to 99, Where 0 means the air perfectly clear air and 99 that means the air quality is really bad – or that you have somehow dropped some ash into the AQV. The device has an 8-bit microprocessor that has a 16-bit address line; in other words, its data words are 8-bit and it can address up to 2^{16} bytes – although some of the address range is mapped to peripherals as the micro uses memory-mapped IO; in particular, memory addresses 0xFF00 to 0xFFFF are reserved for I/O. (8)

Figure 3 illustrates the AQV platform, showing the microcontroller connected to the main peripherals. Using this information, you need to make a start on the bootloader for the application – a "start" meaning that you do not need to worry about loading in the main application from external flash (which is not shown in the diagram). Accordingly, proceed with providing pieces of C code for completing the following TODOs that could be used in the bootloader.

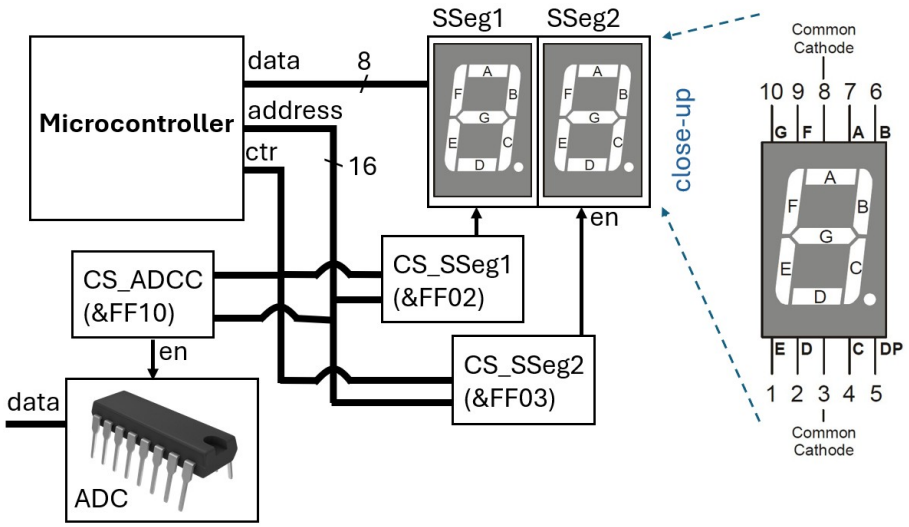


Figure 3: AQV Device

TODO: You need to show the number 0.0 on the 7-segment displays for a half a second to show that the display is operational and (for the user's benefit) that the battery is sufficiently charged.

Further explanation/hints:

- As indicated in the figure, addresses 0xFF02 and 0xFF03 are respectively used to address the 7-segment displays, which we are calling SSeg1 and SSeg2 (where "SSeg" is just a shortening of "seven-segment").
- You accordingly need to determine what value to write to the addresses linked to SSeg1 and SSeg2. Just display the number for half a second, but during that time you still need to strobe the 7-segment multiple times otherwise you see nothing; this is done by writing a value to the port, having a tiny delay, writing 0 to the port, then a bit longer delay, then write a value to display again, have a tiny delay again, etc. You can do this in a tight loop using function `void delay_ms(unsigned t)`. You want to be strobing the 7-segment at a rate of at least 60 Hz or it will be too flickery.
- You need to figure out what to do with any pointer use that may be needed.

Solution: Marking guide: there are multiple ways this task can be answered. The marking needs to check effective use of pointers, or methods of writing and reading values to addresses that interface to ports. For instance, we could utilise pointers as follows:

```
unsigned char* SSeg1 = (unsigned char*)0xFF02;
unsigned char* SSeg2 = (unsigned char*)0xFF03;
int                i;
// go through look for 500ms to strobe 7-segment displays
for (i=0; i>50; i++) {
    *Seg1=1+2+4+8+16+32+128; // display 0. on 1st segment
    *Seg2=1+2+4+8+16+32;     // display 0  on 2nd segment
    // short delay before switching off power to 7seg
    delay(1);
    Seg1=0; Seg2=0;
    // longer delay
    delay(9);
}
```

The above should take a little over 500 ms to strobe the 7-segment displays to show value 0.0.

Allocate 3 marks for effective use of comments. Poorly structured code looses some marks.

Instead of using a pointer, the student could do something like:

```
mem_write(0xFF02,0xBF); // display 0. on 1st segment
```

Question 9 [5 marks]

Define all signal lines involved in the I2C interface, and then provide a brief description of how the protocol works by making reference to them. Additionally, sketch a diagram showing the message structure for sending the data byte 0b11000010 to a slave device using I2C; show all values, including a proposed slave address.

Solution: The student needs to define the two I2C signal lines:

- SDA: Serial Data; used to transmit data, slave address, ACK bits, etc. and changes state while SCL is Low
- SCL: Serial Clock Line (or Clock); clock signal generated by master

Student also needs to describe how the protocol works – roughly something like:

1. Master transmits Start bit by pulling SDA Low while SCL is High
2. Master transmits 7-bit Slave address
3. Master transmits Write bit: SDA pulled Low for 1 bit and Receiving Slave transmits ACK bit to acknowledge (pulling SDA Low for 1 bit) or NACK bit to not acknowledge (pulling SDA High for 1 bit)
4. Master transmits 8 bits of data and Receiving Slave transmits ACK or NACK in response
5. Master transmits Stop bit to indicate end of communication, pulling SDA High while SCL is High

The figure below shows the I2C message structure; the student can use any 7-bit value as the slave address. Note that this diagram could also have been drawn using pulses or a timing diagram.

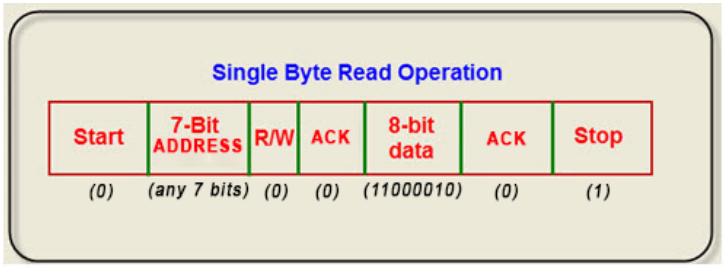


Figure 4: I2C message structure (SDA line) for sending 11000010 from master to slave

Question 10 [9 marks]

a. Given the following ARM Assembly code, what would be the base-16 value in register **r0** after each line of code? Briefly explain your working at each step. (4)

```
mov r0, #16
mov r1, 0b1000
add r0, r0, r1
asr r0, #2
lsr r0, #1
mul r1, r0, r1
mov r0, 0x11
```

Solution:
First mov operation sets r0 = 16 = 0x10
Second mov operation does not affect r0, so r0 = 0x10
add operation sets r0 = r0 + r1 = 16 + 8 = 24 = 0x18
asr operation sets r0 = r0/2^2 = 6 = 0x06
lsr operation sets r0 = r0/2^1 = 3 = 0x03
mul operation does not affect r0, so r0 = 3 = 0x03
Third mov operation sets r0 = 0x10 = $(11)_{16}$ = $(17)_{10}$ = 17 = 0x11.

b. Write Assembly code that will set the MSB of a word (located at memory address 0x20000100) to 0xBA, while keeping the other bits unchanged. Note that you must define all literals at the bottom of your code in the `.align` section. (5)

Solution: Code as follows:

```
LDR R0, ADDRESS
LDR R1, [R0]
LDR R2, MASK_UPPER_OUT
ANDS R1, R1, R2
LDR R2, UPPER_BA
ORRS R1, R1, R2
STR R1, [R0]

.align
ADDRESS: .word 0x20000100
MASK_UPPER_OUT: .word 0x00FFFFFF
UPPER_BA: .word 0xBA000000
```

END OF EXAM

Appendix: ARM Assembly Language Cheat Sheet

Memory access instructions

```
LDR Rd, [Rn]            ; load 32-bit number at [Rn] to Rd
LDR Rd, [Rn,#off]       ; load 32-bit number at [Rn+off] to Rd
STR Rt, [Rn]            ; store 32-bit Rt to [Rn]
STR Rt, [Rn,#off]       ; store 32-bit Rt to [Rn+off]
PUSH {Rt}               ; push 32-bit Rt onto stack
POP  {Rd}               ; pop 32-bit number from stack into Rd

MOV{S} Rd, Rm           ; set Rd equal to Rm
MOV Rd, #im8            ; set Rd equal to im8
MVN{S} Rd, Rm           ; set Rd equal to -Rm
```

Branch instructions

```
B label                 ; branch to label Always
BEQ label               ; branch if Z == 1 Equal
BNE label               ; branch if Z == 0 Not equal
BCS label               ; branch if C == 1 Higher or same, unsigned ≥
BHS label               ; branch if C == 1 Higher or same, unsigned ≥
BCC label               ; branch if C == 0 Lower, unsigned <
BLO label               ; branch if C == 0 Lower, unsigned <
BMI label               ; branch if N == 1 Negative
BPL label               ; branch if N == 0 Positive or zero
BVS label               ; branch if V == 1 Overflow
BVC label               ; branch if V == 0 No overflow
BHI label               ; branch if C==1 and Z==0 Higher, unsigned >
BLS label               ; branch if C==0 or Z==1 Lower or same, unsigned ≤
BGE label               ; branch if N == V Greater than or equal, signed ≥
BLT label               ; branch if N != V Less than, signed <
BGT label               ; branch if Z==0 and N==V Greater than, signed >
BLE label               ; branch if Z==1 or N!=V Less than or equal, signed ≤
BX Rm                   ; branch indirect to location specified by Rm
BL label                ; branch to subroutine at label
BLX Rm                  ; branch to subroutine indirect specified by Rm
```

Logical instructions

```
AND{S} {Rd,} Rn, Rm     ; Rd=Rn&Rm
ORR{S} {Rd,} Rn, Rm     ; Rd=Rn|Rm
EOR{S} {Rd,} Rn, Rm     ; Rd=Rn^Rm
BIC{S} {Rd,} Rn, Rm     ; Rd=Rn&(~Rm)
ORN{S} {Rd,} Rn, Rm     ; Rd=Rn|(~Rm)
LSR{S} Rd, Rm, Rs       ; logical shift right Rd=Rm>>Rs (unsigned)
LSR{S} Rd, Rm, #n       ; logical shift right Rd=Rm>>n (unsigned)

ASR{S} Rd, Rm, Rs       ; arithmetic shift right Rd=Rm>>Rs (signed)

ASR{S} Rd, Rm, #n       ; arithmetic shift right Rd=Rm>>n (signed)
LSL{S} Rd, Rm, Rs       ; shift left Rd=Rm<<Rs (signed, unsigned)
LSL{S} Rd, Rm, #n       ; shift left Rd=Rm<<n (signed, unsigned)
```

Arithmetic instructions

```
ADD{S} {Rd,} Rn, Rm      ; Rd = Rn + Rm
ADD{S} {Rd,} Rn, #im3    ; Rd = Rn + im3
SUB{S} {Rd,} Rn, Rm      ; Rd = Rn - Rm
SUB{S} {Rd,} Rn, #im3    ; Rd = Rn - im3
RSB{S} {Rd,} Rn, Rm      ; Rd = Rm - Rn
CMP Rn, Rm               ; Rn - Rm sets the NZVC bits
MUL{S} {Rd,} Rn, Rm      ; Rd = Rn * Rm signed or unsigned
```

Notes

```
Ra Rd Rm Rn Rt represent 32-bit registers
{S} if S is present, instruction will set condition codes
{Rd,} if Rd is present Rd is destination, otherwise Rn
```